

Android View 中的基本色彩協調功能

來源：<https://codelabs.developers.google.com/harmonize-color-android-views?hl=zh-tw>

步驟數：12

在本程式碼研究室中，您將瞭解如何透過動態主題調整自訂顏色。

目錄

1. [1. 事前準備](#)
2. [2. 應用程式總覽](#)
3. [3. 生成主題](#)
4. [4. 檢查自訂顏色](#)
5. [5. 什麼是色彩協調？](#)
6. [6. 手動調和色彩](#)
7. [7. 正在新增大眾運輸票證](#)
8. [8. 自動協調顏色](#)
9. [9. 自動協調主題屬性](#)
10. [10. 原始碼](#)
11. [11. 使用者介面範例](#)
12. [12. 摘要](#)

步驟 1 · 約 20 分鐘

1. 事前準備

在本程式碼研究室中，您將瞭解如何協調自訂顏色與動態主題產生的顏色。

必要條件

開發人員應

- 熟悉 Android 的基本主題概念
- 熟悉 Android 小工具 Views 及其屬性

課程內容

- 如何使用多種方法在應用程式中套用色彩協調功能
- 色彩調和的運作方式和色彩變化

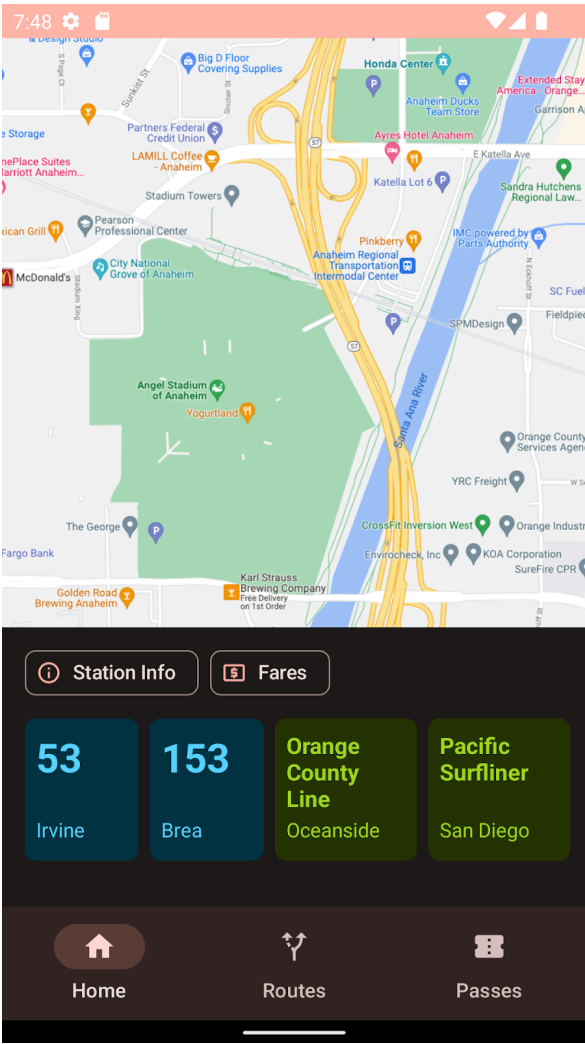
軟硬體需求

- 如果想跟著操作，請使用已安裝 Android 的電腦。
-

步驟 2 · 約 0 分鐘

2. 應用程式總覽

Voyaḡi 是一款已使用動態主題的交通運輸應用程式。對許多大眾運輸系統而言，顏色是火車、公車或輕軌的重要指標，無法以任何可用的動態主要、次要或第三重要顏色取代。我們將著重於彩色大眾運輸資訊卡的 RecyclerView。

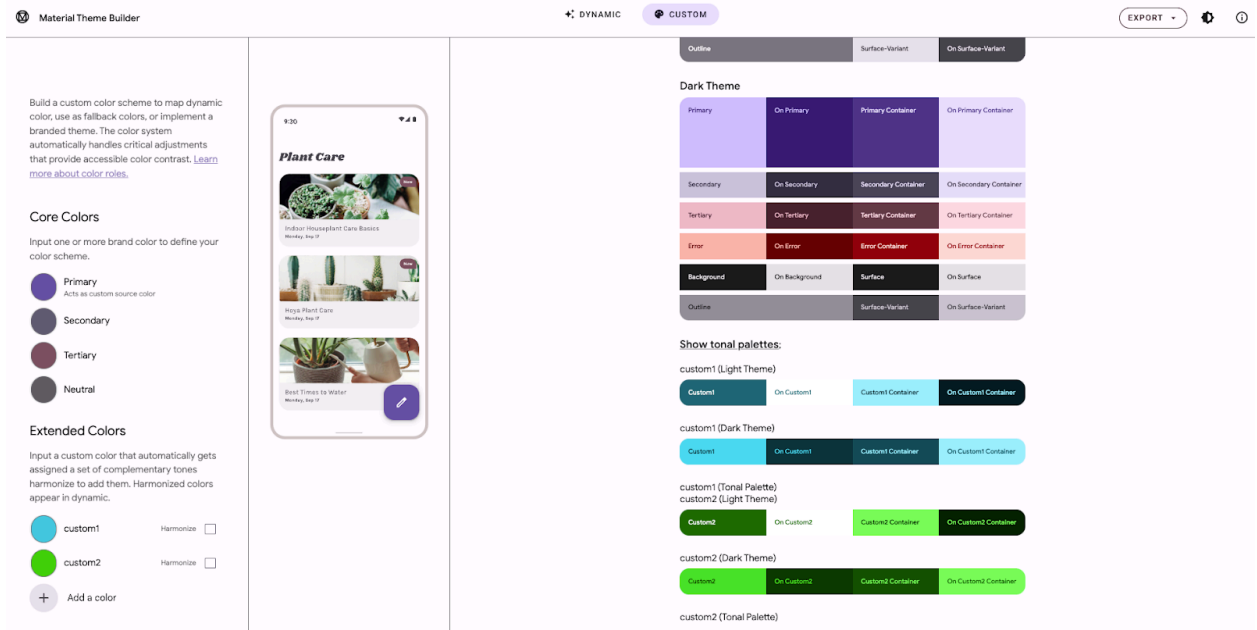


步驟 3 · 約 0 分鐘

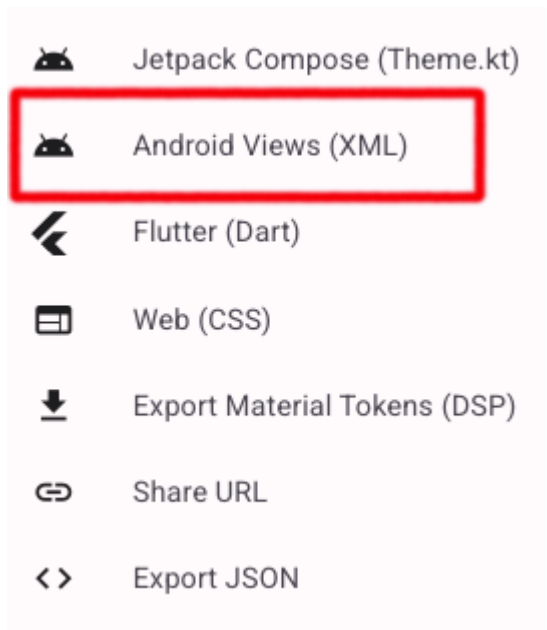
3. 生成主題

建議您先使用 [Material Design 主題設定建構工具](#)，建立 Material 3 主題。在自訂分頁中，您現在可以為主題新增更多顏色。右側會顯示這些顏色的角色和色調調色盤。

在擴充顏色部分，你可以移除或重新命名顏色。



匯出選單會顯示多個可能的匯出選項。撰寫本文時，Material Theme Builder 對協調設定的特殊處理方式僅適用於 Android 檢視區塊



瞭解新的匯出值

無論您是否選擇協調，匯出的下載內容現在都會包含 attrs.xml 檔案，其中含有每個自訂顏色的顏色角色名稱，方便您在主題中使用這些顏色和相關聯的顏色角色。

```

<resources>
    <attr name="colorCustom1" format="color" />
    <attr name="colorOnCustom1" format="color" />
    <attr name="colorCustom1Container" format="color" />
    <attr name="colorOnCustom1Container" format="color" />
    <attr name="harmonizeCustom1" format="boolean" />

    <attr name="colorCustom2" format="color" />
    <attr name="colorOnCustom2" format="color" />
    <attr name="colorCustom2Container" format="color" />
    <attr name="colorOnCustom2Container" format="color" />
    <attr name="harmonizeCustom2" format="boolean" />
</resources>

```

在 `themes.xml` 中，我們為每個自訂顏色產生四個顏色角色 (`color<name>`, `colorOn<name>`, `color<name>Container`, and `colorOn<name>Container`)。 `harmonize<name>` 屬性會反映開發人員是否已在 Material Design 主題設定建構工具中選取選項。不會改變核心主題的顏色。

```

<resources>
    <style name="AppTheme" parent="Theme.Material3.Light.NoActionBar">
        <!-- Normal theme attributes ... -->

        <item name="colorCustom1">#006876</item>
        <item name="colorOnCustom1">#ffffff</item>
        <item name="colorCustom1Container">#97f0ff</item>
        <item name="colorOnCustom1Container">#001f24</item>
        <item name="harmonizeCustom1">>false</item>

        <item name="colorCustom2">#016e00</item>
        <item name="colorOnCustom2">#ffffff</item>
        <item name="colorCustom2Container">#78ff57</item>
        <item name="colorOnCustom2Container">#002200</item>
        <item name="harmonizeCustom2">>false</item>
    </style>
</resources>

```

在 `colors.xml` 檔案中，系統會指定用於產生上述顏色角色的種子顏色，以及顏色調色盤是否會偏移的布林值。

```

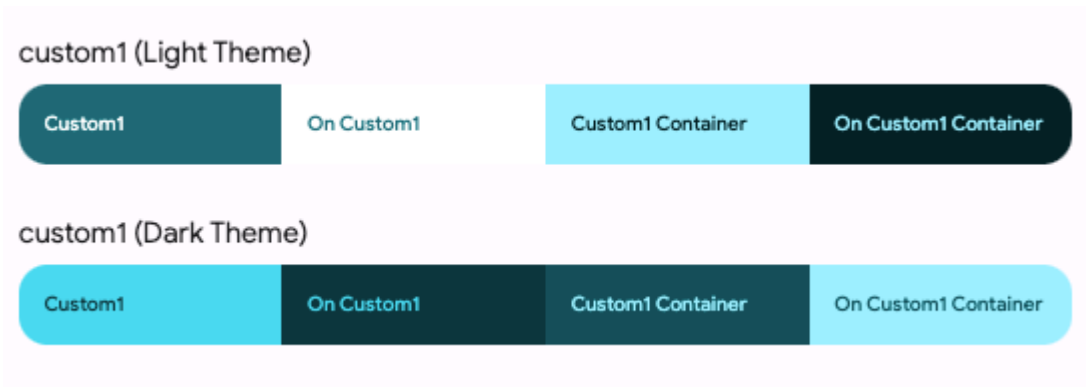
<resources>
    <!-- other colors used in theme -->

    <color name="custom1">#1AC9E0</color>
    <color name="custom2">#32D312</color>
</resources>

```

4. 檢查自訂顏色

放大 Material Design 主題設定建構工具的側邊面板，可以看到新增自訂顏色後，面板會顯示淺色和深色調色盤中的四個主要顏色角色。



在 Android Views 中，我們會為您匯出這些顏色，但這些顏色在幕後可以由 `ColorRoles` 物件的執行個體表示。

`ColorRoles` 類別有四個屬性：`accent`、`onAccent`、`accentContainer` 和 `onAccentContainer`。這些屬性是四種十六進位顏色的整數表示法。

```
public final class ColorRoles {  
  
    private final int accent;  
    private final int onAccent;  
    private final int accentContainer;  
    private final int onAccentContainer;  
  
    // truncated code  
  
}
```

您可以在執行階段使用 `MaterialColors` 類別中的 `getColorRoles`，從任意顏色擷取四個主要顏色角色。`getColorRoles` 可讓您在執行階段建立這組四個顏色角色，並提供特定種子顏色。

```
public static ColorRoles getColorRoles(  
    @NonNull Context context,  
    @ColorInt int color  
) { /* implementation */ }
```

使用 `getColorRoles` 及其輸出內容時請務必小心，顏色輸入參數是已解析的顏色，如 `** getColor(R.color.<name>) ****`。

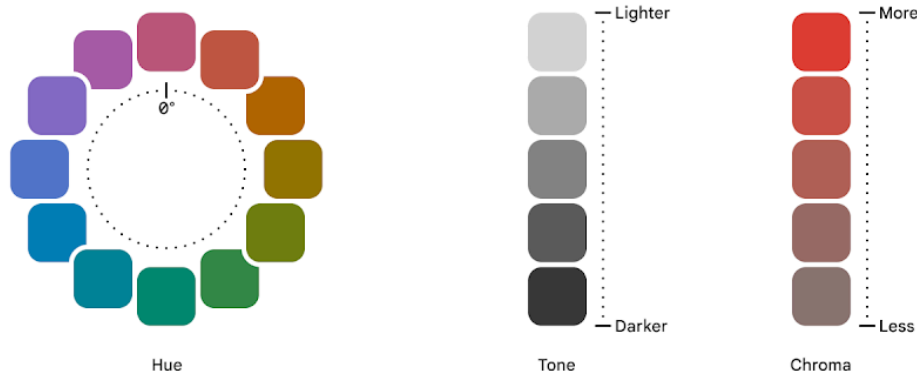
同樣地，輸出值是實際的顏色值，而非指標。`**`

5. 什麼是色彩協調？

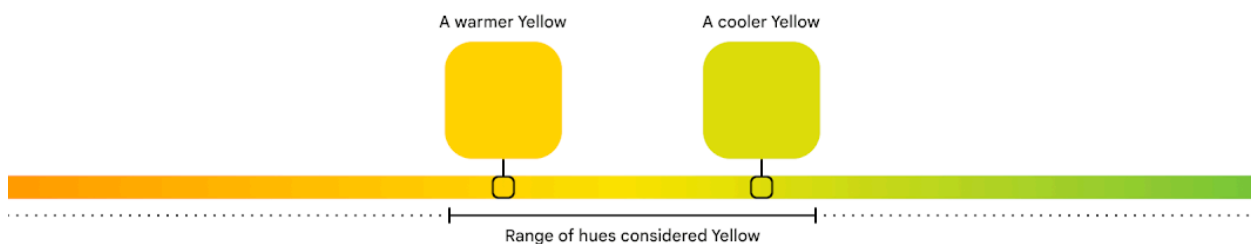
Material 的新色彩系統採用演算法設計，可從指定的種子顏色生成主要、次要、第三重要和中性色。與內部和外部合作夥伴討論時，我們發現大家很關心如何採用動態色彩，同時控管部分顏色。

這些顏色通常在應用程式中帶有特定意義或脈絡，如果替換成隨機顏色，就會失去這些意義。否則這些顏色可能會顯得不協調或不合適。

在 Material You 中，色彩會以色調、色度和色調描述。色調是指顏色在某個色域中與其他顏色相比時，給人的感覺。色調是指色彩的明暗程度，彩度則是色彩的強度。色調的感知可能受到文化和語言因素影響，例如古代文化中常缺乏藍色的字詞，而是將藍色視為與綠色同類。



特定色調可視為暖色或冷色，取決於該色調在色調光譜中的位置。一般來說，偏向紅色、橘色或黃色調會讓影像看起來較暖，偏向藍色、綠色或紫色調則會讓影像看起來較冷。即使是暖色或冷色，也會有暖色調和冷色調。下方「暖色」黃色帶有更多橘色調，而「冷色」黃色則受到綠色影響。



色彩協調演算法會檢查未位移的顏色色調，以及應與其協調的顏色，找出和諧但不會改變其基本色彩品質的色調。在第一張圖片中，光譜上繪製的綠色、黃色和橘色色調較不和諧。在下一個圖片中，綠色和橘色已與黃景色調調和。新版綠色較為暖色，新版橘色則較為冷色。

橘色和綠色的色調已改變，但仍可視為橘色和綠色。



如要進一步瞭解部分設計決策、探索和考量事項，請參閱同事 Ayan Daniels 和 Andrew Lu 撰寫的[網誌文章](#)，其中會比本節更深入探討。

6. 手動調和色彩

如要調和單一色調，`MaterialColors` 中有兩個函式：`harmonize` 和 `harmonizeWithPrimary`。

`harmonizeWithPrimary` 會使用 `Context` 做為存取目前主題的方式，並從中存取主色。

```
@ColorInt
public static int harmonizeWithPrimary(@NonNull Context context, @ColorInt int
colorToHarmonize) {
    return harmonize(
        colorToHarmonize,
        getColor(context, R.attr.colorPrimary, MaterialColors.class.getCanonicalName()));
}

@ColorInt
public static int harmonize(@ColorInt int colorToHarmonize, @ColorInt int
colorToHarmonizeWith) {
    return Blend.harmonize(colorToHarmonize, colorToHarmonizeWith);
}
```

如要擷取這四種音調，我們需要再做一些事。

由於我們已有來源顏色，因此需要執行下列操作：

1. 判斷是否應進行協調，
2. 判斷我們是否處於深色模式，以及
3. 傳回已協調或未協調的 `ColorRoles` 物件。

決定是否要進行協調

在 Material Design 主題設定建構工具匯出的主題中，我們使用 `harmonize<Color>` 命名方式納入布林屬性。以下是存取該值的便利函式。

如果找到，則傳回其值；否則會判斷不應協調顏色。

```
// Looks for associated harmonization attribute based on the color id
// custom1 ==> harmonizeCustom1
fun shouldHarmonize(context: Context, colorId: Int): Boolean {
    val root = context.resources.getResourceEntryName(colorId)
    val harmonizedId = "harmonize" + root.replaceFirstChar { it.uppercaseChar() }

    val identifier = context.resources.getIdentifier(
        harmonizedId, "bool", context.packageName)

    return if (identifier != 0) context.resources.getBoolean(identifier) else false
}
```

建立 Harmonized `ColorRoles` 物件

`retrieveHarmonizedColorRoles` 是另一個便利函式，會合併上述所有步驟：擷取具名資源的顏色值、嘗試解析布林屬性來判斷是否要進行協調，並傳回衍生自原始或混合顏色的 `ColorRoles` 物件 (視淺色或深色配置而定)。

```
fun retrieveHarmonizedColorRoles(
    view: View,
    customId: Int,
    isLight: Boolean
): ColorRoles {
    val context = view.context
    val custom = context.getColor(customId);

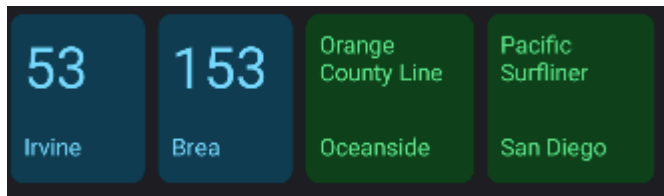
    val shouldHarmonize = shouldHarmonize(context, customId)
    if (shouldHarmonize) {
        val blended = MaterialColors.harmonizeWithPrimary(context, custom)
        return MaterialColors.getColorRoles(blended, isLight)
    } else return MaterialColors.getColorRoles(custom, isLight)
}
```

您可以協調任何一對顏色，不只是與主色。詳情請參閱 [MaterialColors](#) 類別。

步驟 7 · 約 0 分鐘

7. 正在新增大眾運輸票證

如先前所述，我們會使用 RecyclerView 和轉接器填入及著色一系列的大眾運輸票證。



儲存 Transit 資料

為儲存大眾運輸票證的文字資料和顏色資訊，我們使用資料類別儲存名稱、目的地和顏色資源 ID。

```
data class TransitInfo(val name: String, val destination: String, val colorId: Int)

/* truncated code */

val transitItems = listOf(
    TransitInfo("53", "Irvine", R.color.custom1),
    TransitInfo("153", "Brea", R.color.custom1),
    TransitInfo("Orange County Line", "Oceanside", R.color.custom2),
    TransitInfo("Pacific Surfliner", "San Diego", R.color.custom2)
)
```

我們會使用這個顏色，即時產生所需的色調。

您可以使用下列 `onBindViewHolder` 函式，在執行階段進行協調。

```
override fun onBindViewHolder(holder: ViewHolder, position: Int) {
    val transitInfo = list.get(position)
    val color = transitInfo.colorId
    if (!colorRolesMap.containsKey(color)) {

        val roles = retrieveHarmonizedColorRoles(
            holder.itemView, color,
            !isNightMode(holder.itemView.context)
        )
        colorRolesMap.put(color, roles)
    }

    val card = holder.card
    holder.transitName.text = transitInfo.name
    holder.transitDestination.text = transitInfo.destination

    val colorRoles = colorRolesMap.get(color)
    if (colorRoles != null) {
        holder.card.setCardBackgroundColor(colorRoles.accentContainer)
        holder.transitName.setTextColor(colorRoles.onAccentContainer)
        holder.transitDestination.setTextColor(colorRoles.onAccentContainer)
    }
}
```

8. 自動協調顏色

除了手動處理協調作業，您也可以交由系統處理。[HarmonizedColorOptions](#) 是建構工具類別，可讓您指定目前手動執行的許多作業。

擷取目前情境以存取目前的動態配置後，您需要指定要協調的基底顏色，並根據該 `HarmonizedColorOptions` 物件和已啟用 `DynamicColors` 的情境建立新情境。

如不想協調顏色，只要不要將該顏色納入 `harmonizedOptions` 即可。

```
val newContext = DynamicColors.wrapContextIfAvailable(requireContext())

val harmonizedOptions = HarmonizedColorsOptions.Builder()
    .setColorResourceIds(intArrayOf(R.color.custom1, R.color.custom2))
    .build();

harmonizedContext =
    HarmonizedColors.wrapContextIfAvailable(dynamicColorsContext, harmonizedOptions)
```

由於已處理過協調一致的基底顏色，您可以更新 `onBindViewHolder`，直接呼叫 `MaterialColors.getColorRoles`，並指定傳回的角色應為淺色或深色。

```
override fun onBindViewHolder(holder: ViewHolder, position: Int) {
    /*...*/
    val color = transitInfo.colorId
    if (!colorRolesMap.containsKey(color)) {

        val roles = MaterialColors.getColorRoles(context.getColor(color),
!isNightMode(context))

        )
        colorRolesMap.put(color, roles)
    }

    val card = holder.card
    holder.transitName.text = transitInfo.name
    holder.transitDestination.text = transitInfo.destination

    val colorRoles = colorRolesMap.get(color)
    if (colorRoles != null) {
        holder.card.setCardBackgroundColor(colorRoles.accentContainer)
        holder.transitName.setTextColor(colorRoles.onAccentContainer)
        holder.transitDestination.setTextColor(colorRoles.onAccentContainer)
    }
}
```

9. 自動協調主題屬性

目前為止，我們所用的方法都是從個別顏色擷取色彩角色。這很適合用來顯示正在生成真實音調，但對大多數現有應用程式來說並不實際。您可能不會直接衍生顏色，而是使用現有的主題屬性。

在本程式碼研究室的前半部，我們討論了如何匯出主題屬性。

```
<resources>
  <style name="AppTheme" parent="Theme.Material3.Light.NoActionBar">
    <!-- Normal theme attributes ... -->

    <item name="colorCustom1">#006876</item>
    <item name="colorOnCustom1">#ffffff</item>
    <item name="colorCustom1Container">#97f0ff</item>
    <item name="colorOnCustom1Container">#001f24</item>
    <item name="harmonizeCustom1">false</item>

    <item name="colorCustom2">#016e00</item>
    <item name="colorOnCustom2">#ffffff</item>
    <item name="colorCustom2Container">#78ff57</item>
    <item name="colorOnCustom2Container">#002200</item>
    <item name="harmonizeCustom2">false</item>
  </style>
</resources>
```

與第一個自動方法類似，我們可以為 `HarmonizedColorOptions` 提供值，並使用 `HarmonizedColors` 擷取含有協調色彩的 `Context`。這兩種方法的主要差異在於：此外，我們還需要提供主題疊加層，其中包含要協調的欄位。

```
val dynamicColorsContext = DynamicColors.wrapContextIfAvailable(requireContext())

// Harmonizing individual attributes
val harmonizedColorAttributes = HarmonizedColorAttributes.create(
    intArrayOf(
        R.attr.colorCustom1,
        R.attr.colorOnCustom1,
        R.attr.colorCustom1Container,
        R.attr.colorOnCustom1Container,
        R.attr.colorCustom2,
        R.attr.colorOnCustom2,
        R.attr.colorCustom2Container,
        R.attr.colorOnCustom2Container
    ), R.style.AppTheme_Overlay
)
val harmonizedOptions =
    HarmonizedColorsOptions.Builder().setColorAttributes(harmonizedColorAttributes).build()

val harmonizedContext =
    HarmonizedColors.wrapContextIfAvailable(dynamicColorsContext, harmonizedOptions)
```

您的轉接程式會使用協調的環境。主題疊加層中的值應參照未協調的淺色或深色變體。

```

<style name="AppTheme.Overlay" parent="AppTheme">
    <item name="colorCustom1">@color/harmonized_colorCustom1</item>
    <item name="colorOnCustom1">@color/harmonized_colorOnCustom1</item>
    <item name="colorCustom1Container">@color/harmonized_colorCustom1Container</item>
    <item name="colorOnCustom1Container">@color/harmonized_colorOnCustom1Container</item>

    <item name="colorCustom2">@color/harmonized_colorCustom2</item>
    <item name="colorOnCustom2">@color/harmonized_colorOnCustom2</item>
    <item name="colorCustom2Container">@color/harmonized_colorCustom2Container</item>
    <item name="colorOnCustom2Container">@color/harmonized_colorOnCustom2Container</item>
</style>

```

在 XML 版面配置檔案中，我們可以照常使用這些協調屬性。

```

<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
style="?attr/materialCardViewFilledStyle"
android:id="@+id/card"
android:layout_width="80dp"
android:layout_height="100dp"
android:layout_marginStart="8dp"
app:cardBackgroundColor="?attr/colorCustom1Container"
>

<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:layout_margin="8dp">

    <TextView
        android:id="@+id/transitName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="28sp"
        android:textStyle="bold"
        android:textColor="?attr/colorOnCustom1Container"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/transitDestination"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginBottom="4dp"
        android:textColor="?attr/colorOnCustom1Container"
        app:layout_constraintBottom_toBottomOf="parent" />
</androidx.constraintlayout.widget.ConstraintLayout>
</com.google.android.material.card.MaterialCardView>

```


步驟 10 · 約 0 分鐘

10. 原始碼

```

package com.example.voyagi.harmonization.ui.dashboard

import android.content.Context
import android.content.res.Configuration
import android.graphics.Typeface
import android.os.Bundle
import android.util.TypedValue
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.TextView
import androidx.fragment.app.Fragment
import androidx.lifecycle.Observer
import androidx.lifecycle.ViewModelProvider
import androidx.recyclerview.widget.LinearLayoutManager
import androidx.recyclerview.widget.RecyclerView
import com.example.voyagi.harmonization.R
import com.example.voyagi.harmonization.databinding.FragmentDashboardBinding
import com.example.voyagi.harmonization.ui.home.TransitCardAdapter
import com.example.voyagi.harmonization.ui.home.TransitInfo
import com.google.android.material.card.MaterialCardView
import com.google.android.material.color.ColorRoles
import com.google.android.material.color.DynamicColors
import com.google.android.material.color.HarmonizedColorAttributes
import com.google.android.material.color.HarmonizedColors
import com.google.android.material.color.HarmonizedColorsOptions
import com.google.android.material.color.MaterialColors

class DashboardFragment : Fragment() {

    enum class TransitMode { BUS, TRAIN }
    data class TransitInfo2(val name: String, val destination: String, val mode: TransitMode)

    private lateinit var dashboardViewModel: DashboardViewModel
    private var _binding: FragmentDashboardBinding? = null

    // This property is only valid between onCreateView and
    // onDestroyView.
    private val binding get() = _binding!!

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        dashboardViewModel =
            ViewModelProvider(this).get(DashboardViewModel::class.java)

        _binding = FragmentDashboardBinding.inflate(inflater, container, false)
        val root: View = binding.root

```

```

val recyclerView = binding.recyclerView

val transitItems = listOf(
    TransitInfo2("53", "Irvine", TransitMode.BUS),
    TransitInfo2("153", "Brea", TransitMode.BUS),
    TransitInfo2("Orange County Line", "Oceanside", TransitMode.TRAIN),
    TransitInfo2("Pacific Surfliner", "San Diego", TransitMode.TRAIN)
)

val dynamicColorsContext = DynamicColors.wrapContextIfAvailable(requireContext())

// Harmonizing individual attributes
val harmonizedColorAttributes = HarmonizedColorAttributes.create(
    intArrayOf(
        R.attr.colorCustom1,
        R.attr.colorOnCustom1,
        R.attr.colorCustom1Container,
        R.attr.colorOnCustom1Container,
        R.attr.colorCustom2,
        R.attr.colorOnCustom2,
        R.attr.colorCustom2Container,
        R.attr.colorOnCustom2Container
    ), R.style.AppTheme_Overlay
)
val harmonizedOptions =
    HarmonizedColorsOptions.Builder().setColorAttributes(harmonizedColorAttributes).build()

val harmonizedContext =
    HarmonizedColors.wrapContextIfAvailable(dynamicColorsContext, harmonizedOptions)

val adapter = TransitCardAdapterAttr(transitItems, harmonizedContext)
recyclerView.adapter = adapter
recyclerView.layoutManager =
    LinearLayoutManager(harmonizedContext, RecyclerView.HORIZONTAL, false)

return root
}

override fun onDestroyView() {
    super.onDestroyView()
    _binding = null
}
}

class TransitCardAdapterAttr(val list: List<DashboardFragment.TransitInfo2>, context:
Context) :
    RecyclerView.Adapter<RecyclerView.ViewHolder>() {
    val colorRolesMap = mutableMapOf<Int, ColorRoles>()
    private var harmonizedContext: Context? = context

```

```

override fun onCreateViewHolder(
    parent: ViewGroup,
    viewType: Int
): RecyclerView.ViewHolder {
    return if (viewType == DashboardFragment.TransitMode.BUS.ordinal) {
        BusViewHolder(LayoutInflater.from(harmonizedContext).inflate(R.layout.transit_item_bus,
parent, false))
    } else
TrainViewHolder(LayoutInflater.from(harmonizedContext).inflate(R.layout.transit_item_train,
parent, false))
}

override fun getItemCount(): Int {
    return list.size
}

override fun onBindViewHolder(holder: RecyclerView.ViewHolder, position: Int) {
    val item = list[position]
    if (item.mode.ordinal == DashboardFragment.TransitMode.BUS.ordinal) {
        (holder as BusViewHolder).bind(item)
        (holder as TransitBindable).adjustNameLength()
    } else {
        (holder as TrainViewHolder).bind(item)
        (holder as TransitBindable).adjustNameLength()
    }
}

override fun getItemViewType(position: Int): Int {
    return list[position].mode.ordinal
}

interface TransitBindable {
    val card: MaterialCardView
    var transitName: TextView
    var transitDestination: TextView

    fun bind(item: DashboardFragment.TransitInfo2) {
        transitName.text = item.name
        transitDestination.text = item.destination
    }
}

fun Float.toDp(context: Context) =
    TypedValue.applyDimension(
        TypedValue.COMPLEX_UNIT_DIP,
        this,
        context.resources.displayMetrics
    )

fun adjustNameLength(){
    if (transitName.length() > 4) {
        val layoutParams = card.layoutParams
        layoutParams.width = 100f.toDp(card.context).toInt()
        card.layoutParams = layoutParams
        transitName.setTypeface(Typeface.DEFAULT_BOLD);
    }
}

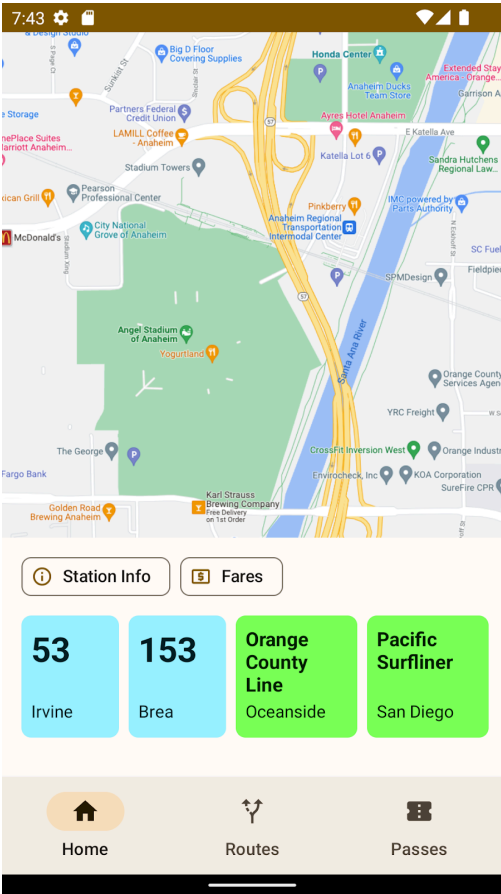
```

```
        transitName.setTextSize(TypedValue.COMPLEX_UNIT_SP, 16.0f)
    }
}

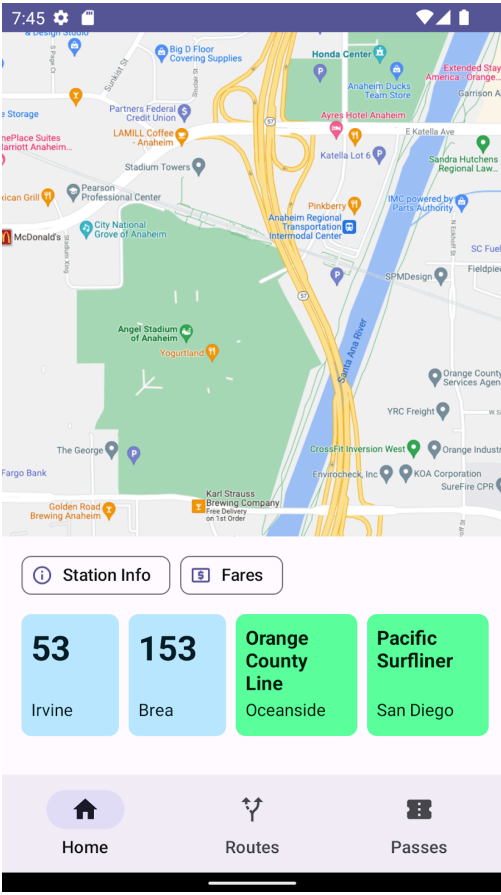
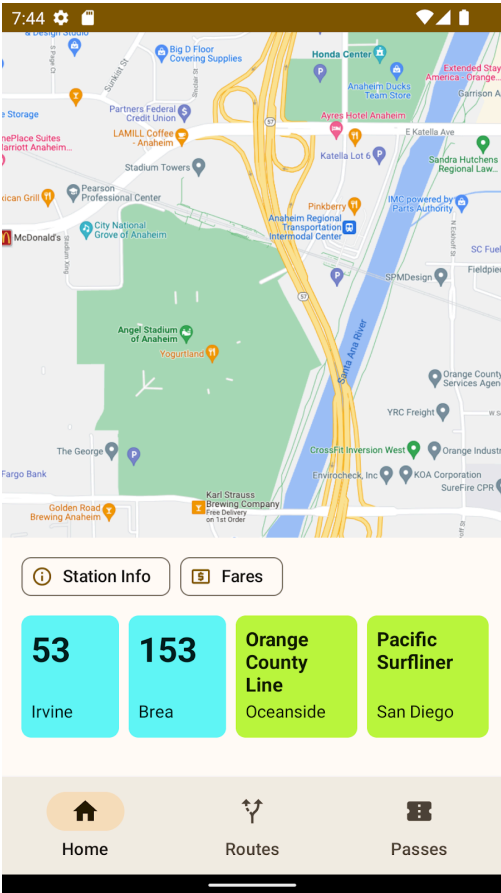
inner class BusViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView),
TransitBindable {
    override val card: MaterialCardView = itemView.findViewById(R.id.card)
    override var transitName: TextView = itemView.findViewById(R.id.transitName)
    override var transitDestination: TextView = itemView.findViewById(R.id.transitDestination)
}
inner class TrainViewHolder(itemView: View) : RecyclerView.ViewHolder(itemView),
TransitBindable {
    override val card: MaterialCardView = itemView.findViewById(R.id.card)
    override var transitName: TextView = itemView.findViewById(R.id.transitName)
    override var transitDestination: TextView = itemView.findViewById(R.id.transitDestination)
}
}
```

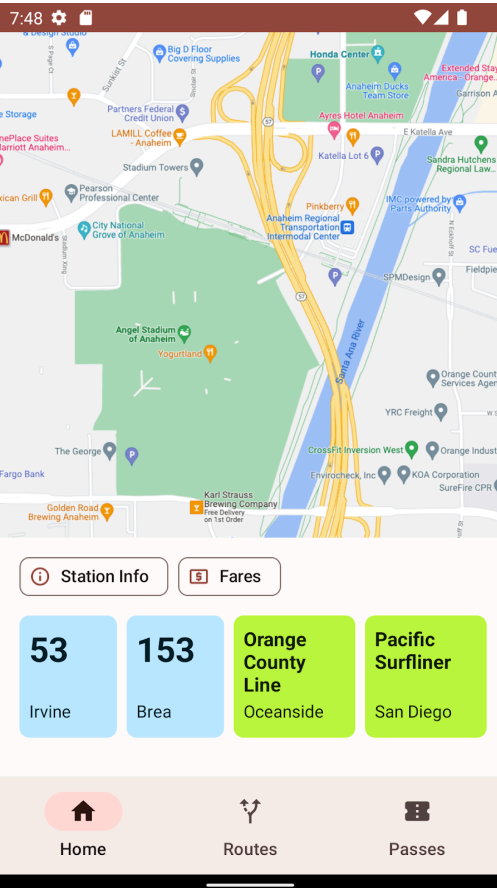
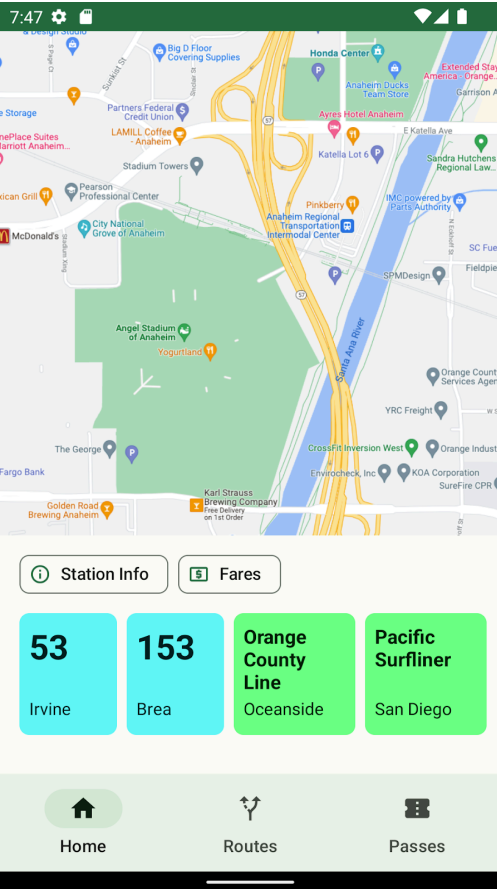
11. 使用者介面範例

預設主題和自訂顏色，不含協調功能



協調的自訂顏色





12. 摘要

在本程式碼研究室中，您已瞭解：

- 色彩協調演算法基本知識
- 如何從指定的可見顏色產生顏色角色。
- 如何選擇性地調和使用使用者介面中的顏色。
- 如何統一主題中的一組屬性。

後續步驟

- 您也可以使用 [Material Design Kit](#)，其中已預先連結元件，因此您可以直接建立主題和連結。
- 瞭解如何透過 [自訂 Material](#)，為使用者提供更個人化的色彩體驗。
- 進一步瞭解新的 [色彩系統](#)。
- 請使用 [Material 主題建構工具](#)！

如有任何疑問，歡迎隨時透過 [Twitter 上的 @MaterialDesign](#) 提問。

更多設計內容和教學課程即將在 youtube.com/MaterialDesign 上線，敬請期待！
